

# Gradient-Product-Balanced Initialization

A Per-Layer Variance Schedule for Row-Centered ReLU Networks

Itay Ofer

April 2026

## Contents

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Goal and Motivation</b>                                        | <b>2</b> |
| <b>2</b> | <b>Forward and Backward Gains for Row-Centered Weights</b>        | <b>2</b> |
| 2.1      | Setup and notation . . . . .                                      | 2        |
| 2.2      | Forward gain: why row centering loses signal . . . . .            | 2        |
| 2.3      | Backward gain: why the backward pass is unaffected . . . . .      | 3        |
| 2.4      | The coupling . . . . .                                            | 3        |
| <b>3</b> | <b>The Product-Balanced Initializer</b>                           | <b>3</b> |
| 3.1      | Derivation . . . . .                                              | 3        |
| 3.2      | Initialization procedure . . . . .                                | 4        |
| <b>4</b> | <b>Per-Layer Variance Scaling: The <math>\eta</math>-Schedule</b> | <b>4</b> |
| 4.1      | Derivation . . . . .                                              | 4        |
| 4.2      | The $\eta$ -parameterization . . . . .                            | 4        |
| <b>5</b> | <b>Empirical Validation (50 Hidden Layers)</b>                    | <b>5</b> |
| 5.1      | Setup . . . . .                                                   | 5        |
| 5.2      | Gain verification . . . . .                                       | 5        |
| 5.3      | Forward gain per layer . . . . .                                  | 5        |
| 5.4      | Backward gain per layer . . . . .                                 | 6        |
| 5.5      | Gain product per layer . . . . .                                  | 6        |
| 5.6      | Gradient row norms . . . . .                                      | 7        |
| <b>6</b> | <b>Geometry Preservation (k-NN Accuracy)</b>                      | <b>7</b> |
| <b>7</b> | <b>The Pareto Frontier</b>                                        | <b>8</b> |
| <b>8</b> | <b>Beyond Row Centering: What Are the Options?</b>                | <b>8</b> |
| 8.1      | The ReLU kernel contraction: an intrinsic limit . . . . .         | 8        |
| 8.2      | Orthogonal He: the current practical winner . . . . .             | 9        |
| 8.3      | Batch normalization: a dynamic alternative . . . . .              | 9        |
| 8.4      | Summary of candidates . . . . .                                   | 9        |
| <b>9</b> | <b>Conclusion</b>                                                 | <b>9</b> |
| 9.1      | Results . . . . .                                                 | 9        |
| 9.2      | Next steps . . . . .                                              | 10       |

# 1 Goal and Motivation

We seek an initialization scheme for deep ReLU networks that simultaneously:

1. **Preserves geometry** — class separability should survive deep projections.
2. **Maintains gradient stability** — gradient magnitudes should be approximately uniform across layers.

From BP4, the gradient of the loss  $\mathcal{C}$  at layer  $\ell$  is:

$$\frac{\partial \mathcal{C}}{\partial W^{(\ell)}} = \frac{1}{n} (\Delta^{(\ell)})^\top A^{(\ell-1)}, \quad (1)$$

where  $A^{(\ell-1)} \in \mathbb{R}^{n \times d_{\ell-1}}$  is the activation matrix (input to layer  $\ell$ ) and  $\Delta^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$  is the error signal matrix. Taking Frobenius norms:

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(\ell)}} \right\|_F \propto \text{rms}(A^{(\ell-1)}) \cdot \text{rms}(\Delta^{(\ell)}). \quad (2)$$

This product depends on two per-layer quantities:

- **Forward gain:**  $g_{\text{fwd}}^{(\ell)} = \text{rms}(A^{(\ell)}) / \text{rms}(A^{(\ell-1)})$  — how the signal changes layer to layer.
- **Backward gain:**  $g_{\text{bwd}}^{(\ell)} = \text{rms}(\Delta^{(\ell)}) / \text{rms}(\Delta^{(\ell+1)})$  — how the error signal changes backward.

We define the **gradient ratio** as  $\max_\ell \|\nabla_{W^{(\ell)}} \mathcal{C}\|_F / \min_\ell \|\nabla_{W^{(\ell)}} \mathcal{C}\|_F$ . A ratio of 1 means perfectly uniform gradients.

## 2 Forward and Backward Gains for Row-Centered Weights

### 2.1 Setup and notation

Consider a layer with row-centered weight matrix  $W \in \mathbb{R}^{d \times d}$ , meaning  $\sum_j W_{ij} = 0$  for every row  $i$ . The per-element variance is  $\text{Var}(W_{ij}) = v/d$ . We define a convenient **variance factor**:

$$s = \sqrt{\frac{v}{2}}. \quad (3)$$

When  $s = 1$ , we recover standard He variance ( $v = 2/d \cdot d = 2$ , i.e.,  $\text{Var}(W_{ij}) = 2/d$ ).

### 2.2 Forward gain: why row centering loses signal

The key question: how much output energy  $\mathbb{E}[(a^{(\ell)})^2]$  does a layer produce, given input energy  $\mathbb{E}[(a^{(\ell-1)})^2]$ ?

**Without row centering** (standard He), the answer is well known. Each neuron computes  $z_i = \sum_j W_{ij} a_j$ , and the variance of  $z_i$  is:

$$\text{Var}(z_i) = d \cdot \text{Var}(W_{ij}) \cdot \mathbb{E}[a_j^2] = v \cdot \mathbb{E}[a^2].$$

After ReLU (which zeroes out  $\sim 50\%$  of values), the output energy is  $\mathbb{E}[(a^{(\ell)})^2] = \frac{v}{2} \cdot \mathbb{E}[(a^{(\ell-1)})^2]$ . He sets  $v = 2$ , giving  $\mathbb{E}[(a^{(\ell)})^2] = \mathbb{E}[(a^{(\ell-1)})^2]$ : forward gain = 1.

**With row centering**, the neuron output changes. Since  $\sum_j W_{ij} = 0$ , we can write:

$$z_i = \sum_j W_{ij} a_j = \sum_j W_{ij} (a_j - \bar{a}), \quad \bar{a} = \frac{1}{d} \sum_k a_k.$$

The constant part  $\bar{a} \cdot \sum_j W_{ij}$  vanishes. The neuron only “sees” the deviations  $a_j - \bar{a}$ , not the mean  $\bar{a}$ .

Now, post-ReLU activations  $a = \text{ReLU}(z)$  with  $z \sim \mathcal{N}(0, \sigma^2)$  have a positive mean:

$$\mathbb{E}[a] = \frac{\sigma}{\sqrt{2\pi}} > 0, \quad \mathbb{E}[a^2] = \frac{\sigma^2}{2}, \quad \text{Var}(a) = \mathbb{E}[a^2] - (\mathbb{E}[a])^2 = \frac{\sigma^2(\pi - 1)}{2\pi}.$$

The fraction of energy in the deviations (what row-centered weights can access) is:

$$\frac{\text{Var}(a)}{\mathbb{E}[a^2]} = \frac{\pi - 1}{\pi} \approx 0.682. \quad (4)$$

This is a universal property of ReLU — independent of  $\sigma$  or  $d$ .

Since row-centered weights see  $\text{Var}(a)$  instead of  $\mathbb{E}[a^2]$ , the output energy becomes:

$$\mathbb{E}[(a^{(\ell)})^2] = \frac{v}{2} \cdot \frac{\pi - 1}{\pi} \cdot \mathbb{E}[(a^{(\ell-1)})^2] = s^2 \cdot \frac{\pi - 1}{\pi} \cdot \mathbb{E}[(a^{(\ell-1)})^2].$$

Taking square roots for the forward gain:

$$\boxed{g_{\text{fwd}} = r \cdot s, \quad r = \sqrt{\frac{\pi - 1}{\pi}} \approx 0.826.} \quad (5)$$

The factor  $r$  is structural: it comes from row centering being blind to the mean of post-ReLU activations. No variance adjustment can remove it.

### 2.3 Backward gain: why the backward pass is unaffected

The backward pass computes  $\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)})$ , where  $\sigma'(z^{(\ell)}) = \mathbf{1}[z^{(\ell)} > 0]$  is the ReLU derivative (binary mask).

Row centering of  $W$  means  $\sum_j W_{ij} = 0$  (rows sum to zero). The transpose  $W^\top$  therefore has  $\sum_i W_{ij} = 0$  (columns sum to zero). By the same logic as the forward pass,  $W^\top$  is blind to the mean of its input  $\delta^{(\ell+1)}$ .

However, unlike post-ReLU activations, the error signal  $\delta$  contains both positive and negative values, so its mean is near zero:  $\text{Var}(\delta)/\mathbb{E}[\delta^2] \approx 1$ . There is almost no “DC component” to lose.

Therefore the backward gain depends only on the weight scale:

$$\boxed{g_{\text{bwd}} = s.} \quad (6)$$

### 2.4 The coupling

Dividing (5) by (6):

$$\frac{g_{\text{fwd}}}{g_{\text{bwd}}} = r \approx 0.826, \quad \text{always, regardless of } s. \quad (7)$$

No single scalar variance can make both gains equal to 1 simultaneously.

## 3 The Product-Balanced Initializer

### 3.1 Derivation

Since we cannot achieve  $g_{\text{fwd}} = g_{\text{bwd}} = 1$ , we seek the next best thing:  $g_{\text{fwd}} \cdot g_{\text{bwd}} = 1$ .

**Theorem 3.1** (Product-balanced variance). *The unique  $s^*$  satisfying  $g_{\text{fwd}} \cdot g_{\text{bwd}} = 1$  is*

$$s^* = \left( \frac{\pi}{\pi - 1} \right)^{1/4} \approx 1.1006, \quad \text{Var}(W_{ij}) = \frac{2\sqrt{\pi/(\pi - 1)}}{d} \approx \frac{2.422}{d}. \quad (8)$$

*Proof.*  $g_{\text{fwd}} \cdot g_{\text{bwd}} = rs \cdot s = rs^2 = 1 \implies s^2 = 1/r = \sqrt{\pi/(\pi - 1)} \implies s^* = (\pi/(\pi - 1))^{1/4}$ .  $\square$

Resulting gains:  $g_{\text{fwd}}^* = ((\pi - 1)/\pi)^{1/4} \approx 0.909$ ,  $g_{\text{bwd}}^* = (\pi/(\pi - 1))^{1/4} \approx 1.101$ ,  $g_{\text{fwd}}^* \cdot g_{\text{bwd}}^* = 1$  (exact).

### 3.2 Initialization procedure

**Definition 3.2** (Product-balanced row-centered initialization). Given a layer with fan-in  $d$ :

1. Set  $\sigma = \sqrt{2\sqrt{\pi/(\pi-1)}} / d$ .
2. Sample  $W_{ij} \sim \mathcal{N}(0, \sigma^2)$ .
3. Row-center:  $W_{ij} \leftarrow W_{ij} - \frac{1}{d} \sum_k W_{ik}$ .
4. Rescale each row:  $\mathbf{w}_i \leftarrow \mathbf{w}_i \cdot \sigma / \text{std}(\mathbf{w}_i)$ .
5. Set bias to zero.

## 4 Per-Layer Variance Scaling: The $\eta$ -Schedule

Uniform variance cannot produce uniform gradients (as we show in Section 7). We derive a per-layer schedule from BP4.

### 4.1 Derivation

**Step 1 (Forward chain).**  $\text{rms}(A^{(\ell-1)}) = \text{rms}(A^{(0)}) \cdot \prod_{k=1}^{\ell-1} (r \cdot s_k) = \text{rms}(A^{(0)}) \cdot r^{\ell-1} \cdot \prod_{k=1}^{\ell-1} s_k$ .

**Step 2 (Backward chain).** The backward gain at position  $k$  is  $g_{\text{bwd}}^{(k)} \approx s_{k+1}$ . Telescoping:  $\text{rms}(\Delta^{(\ell)}) = \text{rms}(\Delta^{(L)}) \cdot \prod_{k=\ell+1}^L s_k$ .

**Step 3 (Gradient at layer  $\ell$ ).** Substituting into (2):

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(\ell)}} \right\| \propto r^{\ell-1} \cdot \frac{\prod_{k=1}^L s_k}{s_\ell}. \quad (9)$$

Since  $\prod_{k=1}^L s_k$  is constant across  $\ell$ :

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(\ell)}} \right\| \propto \frac{r^{\ell-1}}{s_\ell}. \quad (10)$$

**Step 4 (Uniform gradients).**  $r^{\ell-1}/s_\ell = \text{const}$  requires  $s_\ell \propto r^{\ell-1}$ .

### 4.2 The $\eta$ -parameterization

Setting  $s_\ell = C \cdot r^{\ell-1}$  directly would fully correct the gradient, but at the cost of extreme variance differences between layers (since  $r < 1$ , early layers get much larger variance than late layers). We introduce a *correction strength*  $\eta \in [0, 1]$ :

$$s_\ell = s^* \cdot r^{\eta(\ell-(L+1)/2)}, \quad (11)$$

where  $s^* = (\pi/(\pi-1))^{1/4}$  (the product-balanced factor) and the exponent is centered at the middle layer so the geometric mean equals  $s^*$ .

- $\eta = 0$ : uniform variance, no correction. Gradient ratio  $= r^{-(L-1)}$ .
- $\eta = 1$ : full correction, perfect gradient uniformity, but variance ratio  $= r^{-(L-1)}$ .
- $0 < \eta < 1$ : partial correction — a trade-off.

The parameter  $\eta$  is **not derived from theory** — it is a tunable knob that controls the trade-off.

**Definition 4.1** (Layer-balanced product-base initialization). For layer  $\ell \in \{1, \dots, L\}$  with fan-in  $d_{\ell-1}$ :

$$\sigma_\ell = \sqrt{\frac{2}{d_{\ell-1}}} \cdot s^* \cdot r^{\eta(\ell-(L+1)/2)}. \quad (12)$$

Then apply the same procedure as Definition 3.2, using  $\sigma_\ell$  (which varies per layer) instead of a fixed  $\sigma$ .

## 5 Empirical Validation (50 Hidden Layers)

### 5.1 Setup

Fashion-MNIST, 2000 samples, architecture  $784 \rightarrow [784]^{50} \rightarrow 1$ . Four initializers compared:

1. **He** — standard Kaiming (baseline).
2. **RC fwd-balanced** —  $g_{\text{fwd}} = 1$ ,  $g_{\text{bwd}} \approx 1.21$ .
3. **RC product-balanced** —  $g_{\text{fwd}} \cdot g_{\text{bwd}} = 1$  (uniform).
4. **RC layer-balanced (product base,  $\eta = 0.5$ )** — per-layer scaling.

### 5.2 Gain verification

Table 1: Per-layer gain comparison, 50 hidden layers.

| Initializer                                   | Med $g_{\text{fwd}}$ | Med $g_{\text{bwd}}$ | Product      | Grad ratio                   | Grad CV      |
|-----------------------------------------------|----------------------|----------------------|--------------|------------------------------|--------------|
| He (baseline)                                 | 0.993                | 0.997                | 0.990        | $7.8\times$                  | 0.522        |
| RC fwd-balanced                               | 1.000                | 1.209                | 1.210        | $1901\times$                 | 1.903        |
| RC product-balanced                           | 0.909                | 1.099                | 0.999        | $1902\times$                 | 1.906        |
| <b>RC layer-bal ( <math>\eta=0.5</math> )</b> | <b>0.956</b>         | <b>1.049</b>         | <b>1.003</b> | <b><math>36\times</math></b> | <b>1.095</b> |

The layer-balanced variant reduces the gradient ratio from  $1902\times$  (uniform RC) to  $36\times$  — a  $53\times$  improvement. However, He’s  $7.8\times$  remains significantly better.

### 5.3 Forward gain per layer

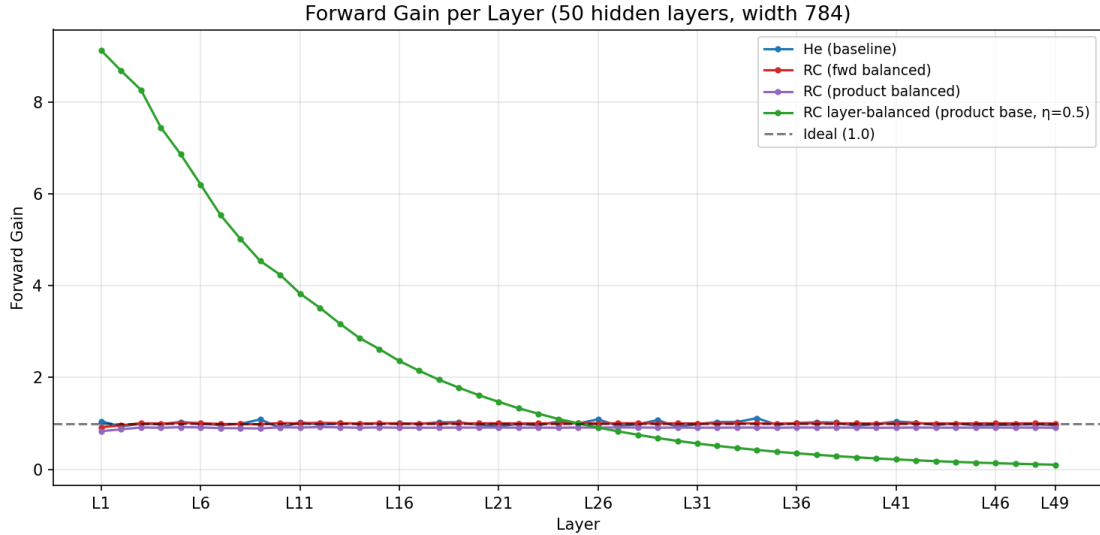


Figure 1: Forward gain per layer. Layer-balanced (green) sits between product-balanced and He.

## 5.4 Backward gain per layer

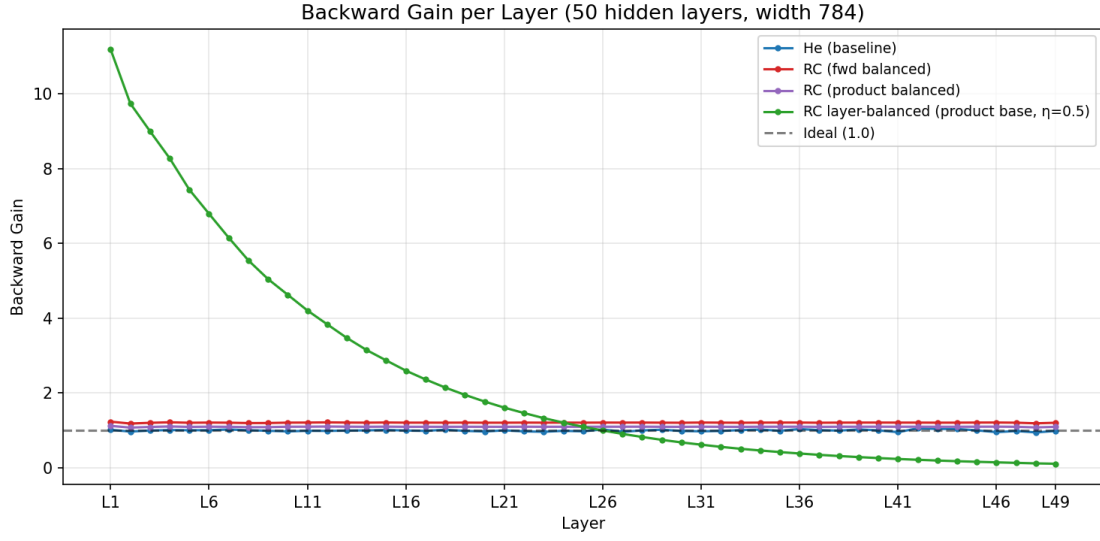


Figure 2: Backward gain per layer. Layer-balanced (green) has moderate backward gain  $\approx 1.05$ .

## 5.5 Gain product per layer

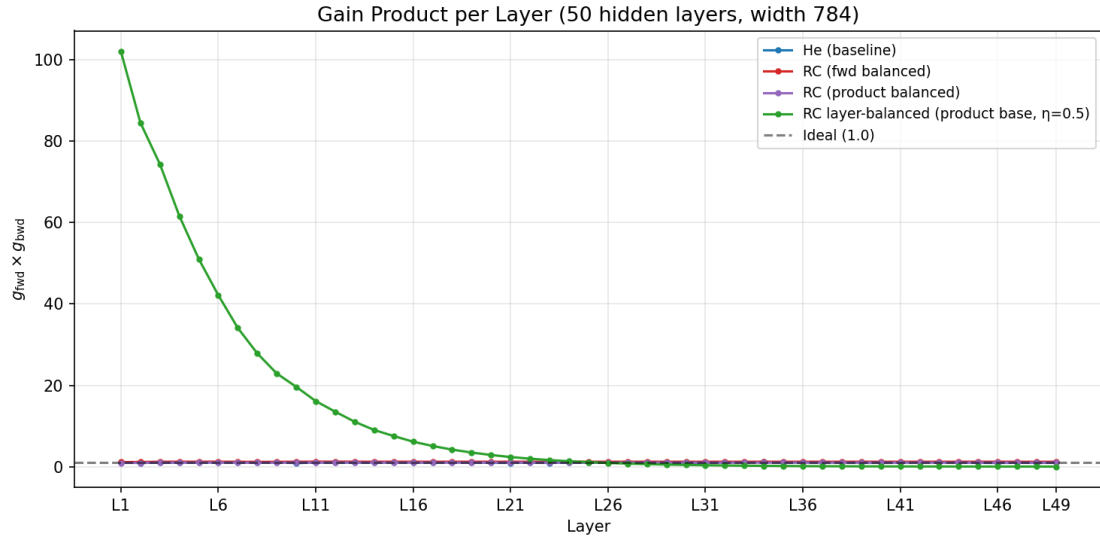


Figure 3: Gain product per layer. Product-balanced and layer-balanced both sit near 1.0.

## 5.6 Gradient row norms

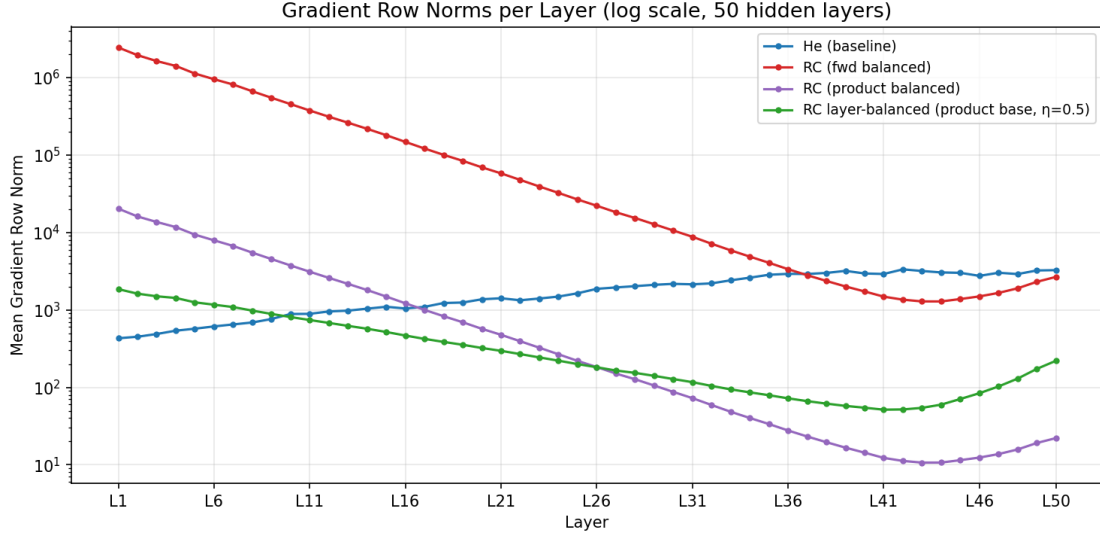


Figure 4: Gradient row norms (log scale). He is nearly flat. Layer-balanced (green) has moderate slope. Uniform RC variants (red, purple) have steep decay.

## 6 Geometry Preservation (k-NN Accuracy)

Table 2: k-NN accuracy vs. depth. Chance level = 0.10.

| Initializer                 | 5 L   | 10 L  | 15 L  | 20 L         |
|-----------------------------|-------|-------|-------|--------------|
| He (baseline)               | 0.829 | 0.800 | 0.784 | <b>0.752</b> |
| RC fwd-balanced             | 0.810 | 0.636 | 0.356 | 0.270        |
| RC product-balanced         | 0.810 | 0.636 | 0.356 | 0.269        |
| RC layer-bal ( $\eta=0.5$ ) | 0.810 | 0.636 | 0.356 | 0.270        |

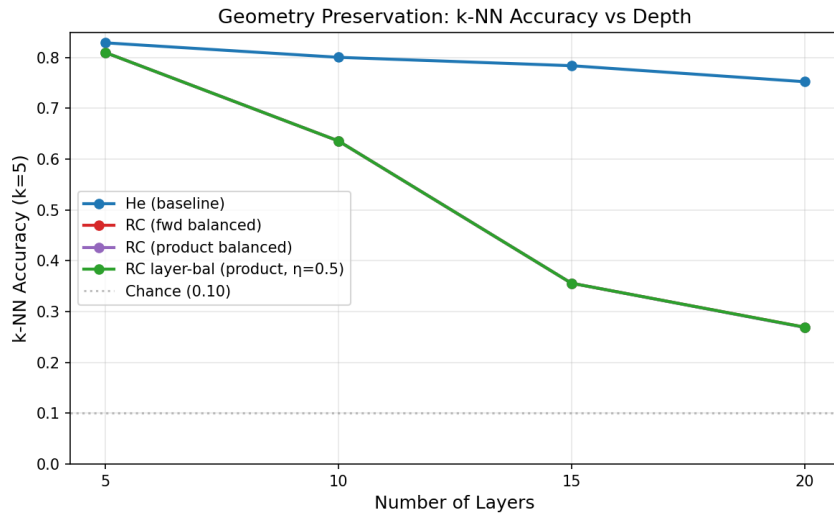


Figure 5: k-NN accuracy vs. depth. All three RC variants are identical.

**Remark 6.1.** All row-centered variants — uniform or layer-balanced, regardless of variance — produce identical k-NN curves. The constraint  $\sum_j W_{ij} = 0$  makes the neuron blind to the mean activation. This DC-blindness is what destroys class structure, and it is completely independent of the variance scale  $s$ .

## 7 The Pareto Frontier

**Theorem 7.1** (Pareto budget). *For the  $\eta$ -schedule (11):*

$$G(\eta) = r^{-(1-\eta)(L-1)} \quad (\text{gradient ratio}), \quad (13)$$

$$V(\eta) = r^{-\eta(L-1)} \quad (\text{variance ratio}). \quad (14)$$

*Their product is independent of  $\eta$ :*

$$\boxed{G(\eta) \cdot V(\eta) = r^{-(L-1)}}. \quad (15)$$

*Proof.*  $G \cdot V = r^{-(1-\eta)(L-1)} \cdot r^{-\eta(L-1)} = r^{-[(1-\eta)+\eta](L-1)} = r^{-(L-1)}$ .  $\square$

Table 3: Pareto budget at various depths.

| Depth $L$ | Budget $r^{-(L-1)}$ |
|-----------|---------------------|
| 10        | $5.6\times$         |
| 20        | $38\times$          |
| 50        | $11,943\times$      |
| 100       | $1.7 \times 10^8$   |

Given a target gradient ratio  $T$ , the required  $\eta$  is:

$$\eta^*(L, T) = \max\left(0, 1 - \frac{\ln T}{(L-1) \cdot \ln(1/r)}\right). \quad (16)$$

## 8 Beyond Row Centering: What Are the Options?

The results above establish that row centering faces two fundamental barriers for deep networks:

1. **Gradient barrier:** the Pareto budget  $r^{-(L-1)}$  grows exponentially with depth.
2. **Geometry barrier:** the DC-blindness ( $\sum_j W_{ij} = 0$ ) destroys class structure independently of variance.

Both barriers stem from the same structural factor  $r = \sqrt{(\pi-1)/\pi}$ . To escape them, we must either change the weight constraint or change the nonlinearity.

### 8.1 The ReLU kernel contraction: an intrinsic limit

Even without row centering, every ReLU layer contracts angles. The arc-cosine kernel gives the expected cosine similarity after one layer:

$$\hat{K}(\alpha) = \frac{1}{\pi}(\sin \alpha + (\pi - \alpha) \cos \alpha), \quad (17)$$

with distortion  $D(\alpha) = \hat{K}(\alpha) - \cos \alpha > 0$  for all  $\alpha \in (0, \pi)$ . This contraction is a property of ReLU itself — no initialization can eliminate it (Chapter 6 of the main report proves that no pointwise nonlinearity achieves  $D(\alpha) = 0$  everywhere).



## 8.2 Orthogonal He: the current practical winner

Orthogonal initialization with He scaling ( $W = \sqrt{2}Q$ , where  $Q$  is from QR decomposition):

- $g_{\text{fwd}} \approx 1$ ,  $g_{\text{bwd}} \approx 1$  (no gradient trap, no coupling).
- Same expected kernel as He — the systematic contraction  $D(\alpha)$  is identical.
- k-NN accuracy: 0.662 at 20 L vs. He’s 0.639 — a  $\sim 3.6\%$  improvement from reduced variance (better concentration around the expected kernel).
- No row centering  $\Rightarrow$  no DC-blindness  $\Rightarrow$  no geometry destruction via the  $71^\circ$  fixed-point mechanism.

The improvement of orthogonal over He is small because it only reduces the *variance* of the kernel estimate, not the *expected* distortion  $D(\alpha)$ .

## 8.3 Batch normalization: a dynamic alternative

The professor’s observation that batch normalization (BN) works well for CNNs but not for fully connected networks is relevant here. BN performs at each layer:

$$\hat{a}_j = \frac{a_j - \mu_B}{\sigma_B},$$

where  $\mu_B$  and  $\sigma_B$  are the batch mean and standard deviation. This is effectively a *data-dependent* centering and rescaling of activations — precisely what row centering tries to achieve via the *weights*.

The critical difference: BN centers the *actual activations* (adapting to the data at each layer), while row centering constrains the *weights* (a fixed structural constraint independent of data). BN can re-center without losing signal because it also rescales; row centering loses signal because the rescaling cannot recover the DC information that was structurally eliminated.

This suggests that the CNN/FC comparison experiments (with and without BN) may reveal whether BN’s dynamic centering avoids the Pareto frontier that static row centering cannot escape.

## 8.4 Summary of candidates

Table 4: Comparison of initialization approaches for deep ReLU networks.

| Approach            | Gradient stability                    | Geometry                 | Limitation            |
|---------------------|---------------------------------------|--------------------------|-----------------------|
| He                  | $g_{\text{fwd}} = g_{\text{bwd}} = 1$ | Degrades (0.64 at 20 L)  | ReLU contraction      |
| Orthogonal He       | $g_{\text{fwd}} = g_{\text{bwd}} = 1$ | Slightly better (0.66)   | Same expected kernel  |
| RC product-balanced | Product = 1                           | Collapses (0.27 at 20 L) | Pareto + DC-blindness |
| RC layer-balanced   | Ratio $36\times$ at $L=50$            | Collapses (0.27 at 20 L) | Same barriers         |
| BatchNorm (dynamic) | Adaptive                              | Unknown (to test)        | Not an initializer    |

# 9 Conclusion

## 9.1 Results

1. The product-balanced variance  $v^* = 2\sqrt{\pi/(\pi-1)}/d \approx 2.422/d$  achieves  $g_{\text{fwd}} \cdot g_{\text{bwd}} = 1$  exactly.
2. Combined with per-layer scaling ( $\eta = 0.5$ ), the gradient ratio at  $L = 50$  improves from  $1902\times$  to  $36\times$ .
3. The Pareto budget  $G(\eta) \cdot V(\eta) = r^{-(L-1)}$  is a fundamental limit that grows exponentially with depth.

4. Geometry (k-NN accuracy) is identical for all RC variants — the variance is irrelevant.
5. Orthogonal He remains the practical winner for geometry + gradients, though it cannot eliminate the intrinsic ReLU kernel contraction.

## 9.2 Next steps

- CNN and FC experiments with and without BatchNorm, using He and candidate initializers, on Fashion-MNIST and CIFAR-10.
- Investigate whether BN’s dynamic centering avoids the Pareto frontier that static row centering cannot escape.